Energy-Aware Routing of Sentiment Analysis Between a Microcontroller and a Transformer Hub

Ehsan Abdulkhader
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
Reg. No. 24BCT0089

Danyson Antony
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
Reg. No. 24BAI0196

Aditya Achar
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
Reg. No. 24BCI0250

***Abstract*—Battery-powered devices such as social robots, wearables and off-grid messengers increasingly need to understand short texts, but a transformer is far too large for a microcontroller. We study a two-tier system in which an ESP32-S3 microcontroller runs a 256 KB hashed n-gram sentiment model and sends only some reviews over Wi-Fi to a transformer on a Raspberry Pi Zero 2 W hub. We compare two ways of choosing what to send: confidence routing, which sends the reviews the device is least sure about, and learned deferral, a second 128 KB model trained to predict whether the hub would correct the device. To avoid inflated results we build a leakage-controlled pool of 4.1 million IMDb reviews that is movie- and author-disjoint from the official test set, and we choose every threshold on validation data. The device model reaches 92.7 percent test accuracy, and hubs trained on one million extra reviews reach 95.8 percent (DistilBERT) and 96.7 percent (RoBERTa-base). Across three hubs, learned deferral never differs from confidence routing by more than 0.14 accuracy points at the same share of reviews sent, and no difference is statistically significant after correcting for multiple tests. A stronger hub rarely breaks a correct device answer, which removes most of the advantage learned deferral could have. With a datasheet-based energy model, sending 8 to 17 percent of reviews closes half to three quarters of the device-hub accuracy gap at 6 to 12 times less device energy than sending every review.**

***Keywords*—sentiment analysis, model cascades, learning to defer, TinyML, energy-aware inference, edge computing**

I. Introduction

Many battery-powered devices handle text or speech: social and assistive robots that react to what people say [1], off-grid messengers on which people type straight into an ESP32-S3 [2], and wearables paired with a phone. On these devices computation is a real battery cost. Measured on mobile robots, motors account for only 16.6–54.6% of total power [3], and larger onboard accelerators drain robot batteries hours faster [4]. A transformer such as DistilBERT [5] needs hundreds of megabytes, far more than the 512 KB of on-chip SRAM of an ESP32-S3 [6], while a bag-of-n-grams model fits in a few hundred kilobytes but is several points less accurate.

A natural design is a cascade [7], [8]: a tiny model runs on the device and hands hard inputs over a radio link to a transformer on a nearby hub, here a Raspberry Pi Zero 2 W with 512 MB of RAM [9]. Every hand-off costs radio energy, so the central question is which inputs to send. The usual rule sends the inputs on which the device model is least confident [10], [11]. Learning to defer [12], [13] suggests a better rule: train a model to predict whether the hub would actually *fix* the device's answer, so that the device stops sending inputs that the hub would also get wrong or that the device already gets right.

This paper tests that idea on IMDb movie-review sentiment [14] under a strict evaluation protocol. Our contributions are:

- a leakage-controlled training pool of 4.1 million IMDb reviews from which copies, near-duplicates, reviews of test-set movies and reviews by test-set authors are removed, so that results on the official test set are not inflated;
- a microcontroller-sized sentiment model (256 KB of int8 hashed n-gram weights) that reaches 92.69% test accuracy, and transformer hubs trained on one million extra reviews that reach 95.83% (DistilBERT) and 96.72% (RoBERTa-base);
- a controlled comparison of learned deferral against confidence routing across three hubs, with all thresholds chosen on validation data and paired significance tests on the test set. Learned deferral does not beat confidence routing, and we show why;
- an energy model of the device, based on datasheet currents, that turns routing curves into accuracy versus energy per review.

II. Related Work

A. Cascades and Edge-Cloud Inference

Cheap-then-expensive cascades go back to the Viola–Jones detector [7]. Early-exit networks let easy inputs leave a network early [15], and distributed networks split one model across device, edge and cloud [16]. In NLP, confidence-based cascades trade accuracy for compute [8], and edge-cloud cascades with statistical guarantees have been proposed for images and question answering [17]. In our setting the first stage is a microcontroller with no neural network at all, and the cost to minimize is device energy rather than latency.

B. Selective Classification and Learning to Defer

Selective classification abstains when confidence is low [10] and relies on calibrated confidences [11]. Learning to defer trains a rule that passes inputs to a downstream expert [12], with consistent surrogate losses for joint training [13]. Most work defers to a human or a large expert and trains the classifier and the rule jointly. Here the device model is fixed and the deferral model itself must fit beside it on the device, in 128 KB.

C. TinyML and the Energy Cost of NLP

TinyML surveys describe the memory and energy limits of microcontrollers and the trade-off between on-device processing and offloading [18], [19]; energy-aware multi-exit models make local-versus-offload decisions for zero-energy devices [20]. NLP energy has mostly been studied for training [21], [22] or for transformer inference on custom accelerators [23]. Transformers shrunk to microcontroller size lose too

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

much accuracy [24], which is why our device uses hashed n-grams [25] with Naive Bayes feature scaling [26], an approach that scales well with training data [27].

D. IMDb Accuracy and Leakage

About 2.9% of IMDb test labels are wrong [28], so roughly 97% is a practical ceiling; the best peer-reviewed result is 97.1% [29]. Large language models show signs of having seen the IMDb test set [30], so we use none. Large scraped review collections [31] contain copies of test reviews, which motivates the cleaning in Section III-B.

III. System and Method

A. System Overview

Fig. 1 shows the system. For each review x the device computes its own probability of a positive review, $p(x)$, and a deferral score $g(x)$. It sends the review to the hub only if $g(x) > \lambda$; otherwise it answers locally. The threshold λ sets the operating point, i.e., the share of reviews sent, and in an energy-aware deployment it follows from the battery budget. Accuracy is measured offline with the exact int8 arithmetic the device runs; energy comes from the model in Section III-F, because hardware measurements are still in progress.

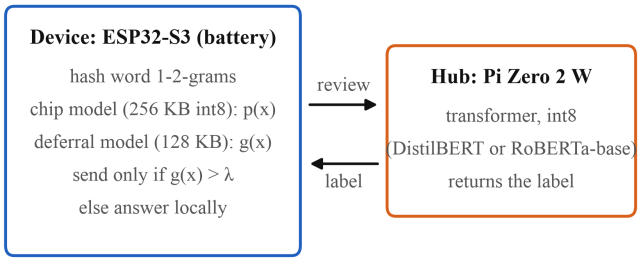


Fig. 1. System overview. The device answers locally unless the deferral score exceeds the threshold λ, in which case it sends the review over Wi-Fi to the hub.

B. Data and Leakage Control

Labels follow the IMDb rule of [14]: ratings of 4/10 or less are negative and 7/10 or more are positive (for Amazon, 1–2 stars and 4–5 stars). The official 25,000-review training split is divided into 20,000 training and 5,000 validation reviews with a fixed seed. The official 25,000-review test split is used only to report final numbers.

The extra-data pool starts from 5.5 million scraped IMDb reviews [31] and 17.3 million Amazon Movies & TV reviews [32], which leave 4.1 million and 12.0 million labelled, de-duplicated reviews. A first cleaning round removed 23,818 pool copies of validation or test reviews. A second round flags reviews of test-set movies (163,409, matched by title and year), all 635,853 reviews by the 16,819 authors who also wrote a test review, and 443 near-duplicates that share at least 30% of a validation or test review's word 8-grams. The remaining 3,336,925 IMDb reviews are split into a balanced training slice (1,532,574), a held-out deferral slice (300,000) and a 50,000-review *val_pool* drawn from a held-out 5% of movies, used to choose model checkpoints. Neither the device model nor any hub trains on the deferral slice; we verified that none of its reviews appear in the hub training data.

C. On-Device Model

The device model hashes word unigrams and bigrams into 2^{18} buckets [25] with binary, L2-normalized features. Naive Bayes log-count ratios [26] are folded into the weights, which are trained with logistic loss and averaged SGD for five epochs and then quantized to int8 with one float scale: 256 KB in total. Inference is a sum of looked-up int8 weights, so no vocabulary is stored. Regularization is chosen on validation data. We use two versions: *v1*, trained on 1.94 million reviews after the first cleaning round (92.69% test), and *v2*, trained on the 1.55 million reviews left after the second round (92.32% test). The v2 model is used with the new hubs, because its training data is disjoint from the re-drawn deferral slice.

D. Hub Models

The hubs are DistilBERT-base-uncased [5] and RoBERTa-base [33]. Reviews longer than 512 tokens keep their first 128 and last 382 tokens [34]. The baseline hub is DistilBERT fine-tuned for two epochs on the 20,000 official training reviews. The new hubs are fine-tuned for one epoch on those reviews plus the first 1,000,000 reviews of the training slice, with batch size 32, AdamW, peak learning rate 3×10^{-5} (DistilBERT) or 2×10^{-5} (RoBERTa-base), linear warm-up over 6% of steps, length-bucketed batches and bfloat16 mixed precision on one NVIDIA H100. The checkpoint with the best val_pool accuracy, evaluated every 2,000 steps, is kept. Both runs, with tokenization, cost USD 2.56 of cloud compute. As int8 models they should need about 67 MB and 125 MB, which fits the hub's 512 MB of RAM; RoBERTa-base needs roughly twice DistilBERT's compute per review.

E. Routing Rules

Confidence routing sends the reviews with the highest score $1 - \max(p, 1 - p)$. *Learned deferral* uses the deferral slice, where both models' answers are known, to define the gain of sending a review:

$$\text{gain}(x) = \mathbf{1}[\text{hub correct}] - \mathbf{1}[\text{device correct}], \qquad (1)$$

which is −1, 0 or +1. A three-class logistic regression predicts the gain from features the device can compute: 2^{16} hashed n-gram buckets, the device's confidence in ten one-hot bins and the review length in nine one-hot bins. The deferral score is the expected accuracy gain of sending,

$$g(x) = P(\text{gain} = +1 \mid x) - P(\text{gain} = -1 \mid x). \qquad (2)$$

The model trains on 250,000 reviews of the deferral slice; its regularization strength is chosen from {0.1, 0.3, 1, 3, 10} by accuracy at 10% sent on the other 50,000. On the device the score needs only two int8 weight tables, the fix and break logits relative to no change, so the router takes 128 KB and the two device models together use 384 KB of the ESP32-S3's 512 KB of SRAM. A *perfect router*, which sends exactly the reviews the hub fixes, gives an upper bound.

F. Energy Model

The device energy per review when a share s of reviews is sent is

$$E(s) = V\,[I_{\text{cpu}}\,t_{\text{local}} + s\,(I_{\text{tx}}\,t_{\text{tx}} + I_{\text{rx}}\,t_{\text{wait}})]. \qquad (3)$$

Currents come from the ESP32-S3 datasheet [6]: $V = 3.3$ V, $I_{\text{cpu}} = 51.2$ mA (240 MHz, one core), $I_{\text{tx}} = 283$ mA (802.11n MCS7 at 18.5 dBm) and $I_{\text{rx}} = 88$ mA. The timings are placeholders until measured: $t_{\text{local}} = 5$ ms, $t_{\text{tx}} = 10$ ms and a hub wait $t_{\text{wait}} = 1$ s with the radio listening. This gives 0.84 mJ per review when nothing is sent and 300.6 mJ when everything is sent. Idle and sleep power are excluded. Because t_{wait}

dominates, energy is nearly proportional to s, so ratios between operating points barely depend on the exact timings.

G. Evaluation Protocol

For a target share t of reviews sent, a router's threshold is the $(1 - t)$ quantile of its scores on the 5,000 validation reviews and is applied unchanged to the test set, so test send shares differ slightly from t. We also report the send share with the highest validation accuracy. Routers are compared with an exact McNemar test on the 25,000 paired test predictions. With five operating points for each of three hubs (15 tests), the Bonferroni-corrected significance level is 0.0033.

IV. Results

A. Component Models

Table I lists every model alone. More data helps the device model steadily: validation accuracy rises from 90.18% with 20,000 reviews to 92.76% with 1.94 million. Two ablations show that the device model is not inflated by leakage or helped by other domains: adding reviews of test-set movies changes test accuracy by +0.02 points, and adding 1.25 million long Amazon reviews changes it by −0.13. One million extra reviews lift DistilBERT by 2.3 points. For reference, a stacked ensemble of ELECTRA-large [35], DeBERTa-v3-large [36] and the 1 MB n-gram model reaches 97.40% (95% confidence interval 97.21–97.58%), above the best peer-reviewed 97.1% [29] in the extra-training-data setting; it is far too large for the hub and serves only as an accuracy ceiling.

TABLE I. Models Alone on the Official IMDb Test Set (25,000 Reviews)

Model	Size	Training reviews	Test accuracy (%)
TF-IDF + logistic regression	full vocab.	20k	90.28
Device v1, int8 n-grams	256 KB	1.94M	92.69
Device v2, int8 n-grams	256 KB	1.55M	92.32
Device v2, int8 n-grams	1 MB	1.55M	92.80
DistilBERT	66M param.	20k	93.51
DistilBERT	66M param.	1.02M	95.83
RoBERTa-base	125M param.	1.02M	96.72
RoBERTa-large	355M param.	1.02M	97.21
Ensemble (reference only)	≈0.8B param.	1.02M	97.40

Device v1 and v2 differ in the cleaning round of their training data (Section III-B).

B. Learned Deferral Versus Confidence Routing

Table II compares the two routers at the same target share of reviews sent, with thresholds chosen on validation data, and Fig. 2 shows the full test curves. With the weak 20k-review hub, routing beats both models: confidence routing reaches 94.05% at 10.7% sent, above the hub alone (93.51%), because that hub breaks almost as many device answers as it fixes. Across all three hubs, however, learned deferral stays within −0.12 to +0.14 points of confidence routing. Only two of the 15 comparisons have $p < 0.05$, and they point in opposite directions: with the weak hub at 10% sent learned deferral is worse (−0.12, $p = 0.04$), and with RoBERTa-base at 10% sent it is better (+0.10, $p = 0.03$). Neither survives the Bonferroni correction. The RoBERTa-base difference is partly a send-share effect: its learned threshold sends 10.7% of test reviews against 10.5% for confidence routing, and at exactly 10% sent on the test curve the gap is +0.04 points. At the send share with the best validation accuracy, confidence routing is ahead with the weak hub (94.07% at 11.5% sent versus 93.99% at 16.7%, $p = 0.22$); with the strong hubs validation accuracy keeps rising, so both routers send over half of all reviews and end within 0.04 points of each other. A variant of the deferral model that sees only the confidence and length bins stays within 0.07 points of confidence routing at 10% sent, so the 128 KB of hashed n-gram weights add little.

TABLE II. Test Accuracy (%) at a Target Share of Reviews Sent; Thresholds Chosen on Validation Data

Hub (alone)	Router	5%	10%	20%	30%
DistilBERT, 20k (93.51)	Confidence	93.63	94.05	93.91	93.81
	Learned	93.64	93.92	93.98	93.95
	McNemar p	0.94	0.04	0.28	0.10
DistilBERT, +1M (95.83)	Confidence	93.71	94.70	95.48	95.72
	Learned	93.68	94.75	95.54	95.72
	McNemar p	0.64	0.24	0.07	1.00
RoBERTa-base, +1M (96.72)	Confidence	93.97	95.14	96.12	96.52
	Learned	93.91	95.23	96.16	96.52
	McNemar p	0.37	0.03	0.28	1.00

The weak hub uses device v1 and the original deferral slice; the +1M hubs use device v2 and the re-drawn slice. Test send shares are within 2 points of the target.

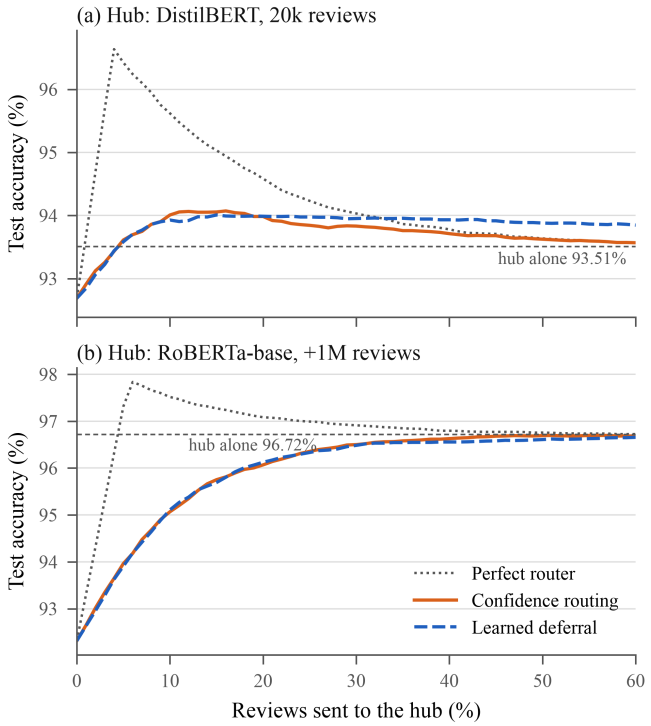


Fig. 2. Test accuracy versus the share of reviews sent to the hub for (a) the weak 20k-review hub and (b) RoBERTa-base trained with one million extra reviews. The learned deferral and confidence routing curves nearly coincide; the perfect router shows the headroom.

C. Why Learned Deferral Does Not Help

Learned deferral can beat confidence routing in two ways: by not sending reviews the hub would break, and by finding the reviews the hub would fix. On the deferral slice the weak hub fixes 4.1% of device answers but breaks 3.9%; with one million extra reviews breaks fall to 1.5% (DistilBERT) and 1.2% (RoBERTa-base), while fixes rise to 5.0% and 5.4%. A stronger hub therefore removes most of the first advantage. At 10% sent with RoBERTa-base, only 4.6–4.8% of the reviews either router sends are broken, but 58% are reviews the device already gets right; learned deferral does not reduce that share (58.2% versus 58.0%). The second advantage requires knowing which reviews the hub fixes, and the hashed n-grams carry little information about that beyond the device's own

confidence. The headroom is real: a perfect router reaches 97.32% at 5% sent with either 1M-review hub, against 93.68–93.97% for the practical routers.

D. Energy

Table III converts operating points into device energy. Each target closes a share of the device-hub gap on validation data; the corresponding threshold is then applied to test. With the 1M-review hubs, sending 8.3% of reviews closes half the gap at 25.8 mJ per review, about 12× less than sending everything, and sending 15–17% closes three quarters at 6–7× less. The weak hub looks far cheaper (up to 70×) only because it barely beats the device. Matching a strong hub exactly is expensive: on the test curves, confidence routing first reaches the hub's accuracy at 43% sent for DistilBERT and 92% for RoBERTa-base. Fig. 3 shows the trade-off: below about 10 mJ per review all hubs perform alike because most answers come from the device, while above about 20 mJ the stronger hubs give clearly more accuracy for the same energy. Because RoBERTa-base needs roughly twice DistilBERT's hub compute, its true per-send energy will be higher once t_{wait} is measured.

TABLE III. DEVICE ENERGY PER REVIEW WITH CONFIDENCE ROUTING (HUB WAIT 1 S); TARGETS CHOSEN ON VALIDATION DATA

Hub	Gap closed	Sent (test)	Test acc. (%)	mJ / review	Less than send-all
DistilBERT 20k	50%	1.2%	92.93	4.3	69.7×
DistilBERT 20k	75%	2.1%	93.14	7.2	41.8×
DistilBERT 20k	90%	3.4%	93.31	11.1	27.0×
DistilBERT 1M	50%	8.3%	94.41	25.8	11.7×
DistilBERT 1M	75%	15.0%	95.19	45.9	6.5×
DistilBERT 1M	90%	26.1%	95.68	79.0	3.8×
RoBERTa-base 1M	50%	8.3%	94.76	25.8	11.7×
RoBERTa-base 1M	75%	16.9%	95.89	51.5	5.8×
RoBERTa-base 1M	90%	25.1%	96.36	76.0	4.0×

Sending every review costs 300.6 mJ; answering every review on the device costs 0.84 mJ. Timings are placeholders until measured.

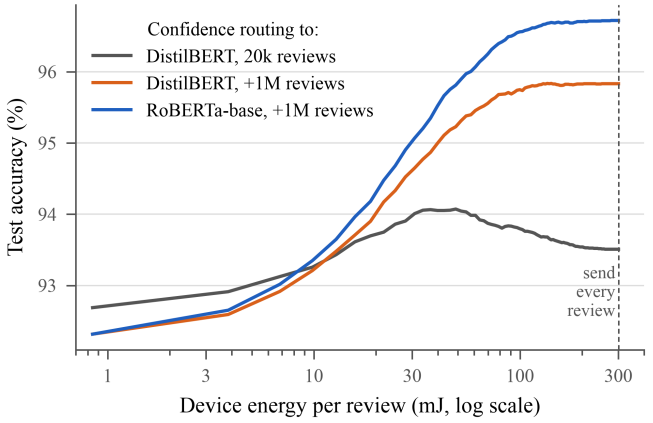


Fig. 3. Test accuracy versus modelled device energy per review for confidence routing to each hub. The dashed line marks sending every review.

V. DISCUSSION AND LIMITATIONS

Energy is modelled, not measured. Currents come from the datasheet and the timings are placeholders; Wi-Fi association, idle and sleep power, and the hub's own power are excluded, and the hub latency and int8 model sizes on the Pi Zero 2 W are estimates. Firmware for the ESP32-S3 and measurements with an INA219 current sensor are the next step.

Leakage matters for deferral labels too. An earlier deferral slice, drawn before the second cleaning round, shares 31% of its reviews (94,194 of 300,000) with the new hubs' training data. Labels from that slice would have made the hub look better on it than it is, so we re-drew the slice.

Statistical power is limited. A difference of 0.1 points is 25 of the 25,000 test reviews, so small router improvements are hard to establish on IMDb; a deferral method would need a clearly larger effect to be worth its 128 KB.

Scope. We study one dataset of long English reviews and router features that fit on the device. Richer signals, such as disagreement between two small device models, may capture which reviews the hub fixes; shorter messages, as on off-grid devices, may also behave differently.

VI. CONCLUSION

We built a microcontroller-to-hub sentiment cascade under a leakage-controlled protocol and asked whether learning to defer chooses better what to send than device confidence does. It does not: across three hubs from 93.5% to 96.7% accuracy, learned deferral stays within 0.14 points of confidence routing and no difference survives correction for multiple tests. Stronger hubs rarely break device answers, which leaves only the hard problem of predicting which reviews the hub will fix. Confidence routing is therefore the practical choice for such devices: with a datasheet-based energy model, sending one review in six closes three quarters of the device-hub gap at 6–7× less device energy than sending every review. Measuring those energies on real hardware is the remaining step.

ACKNOWLEDGMENT

Hub and reference models were trained on Modal cloud GPUs. All code, data-cleaning steps and prediction files are in the project repository.

REFERENCES

[1] M. Spezialetti, G. Placidi, and S. Rossi, "Emotion recognition for human-robot interaction: Recent advances and future perspectives," *Frontiers in Robotics and AI*, vol. 7, Art. no. 532279, 2020.

[2] LILYGO, "T-Deck: ESP32-S3 handheld with keyboard and LoRa radio," product listing. [Online]. Available: https://store.rokland.com/products/lilygo-t-deck-cypher-complete-edition-w-4000-or-8000-mah-battery-gps-keyboard-touchscreen

[3] L. Liu, W. Shi, and K. G. Shin, "Power-efficient autonomous mobile robots," arXiv:2511.20467, 2025.

[4] S. Pohland *et al.*, "Offload or overload: A platform measurement study of mobile robotic manipulation workloads," arXiv:2603.18284, 2026.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[6] Espressif Systems, "ESP32-S3 series datasheet," v2.2. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf

[7] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, vol. 1, 2001, pp. I-511–I-518.

[8] N. Varshney and C. Baral, "Model cascading: Towards jointly improving efficiency and accuracy of NLP systems," arXiv:2210.05528, 2022.

[9] Raspberry Pi Ltd, "Raspberry Pi Zero 2 W product brief." [Online]. Available: https://pip.raspberrypi.com/documents/RP-008359-DS-raspberry-pi-zero-2-w-product-brief.pdf

[10] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," arXiv:1705.08500, 2017.

[11] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” arXiv:1706.04599, 2017.

[12] D. Madras, T. Pitassi, and R. Zemel, “Predict responsibly: Improving fairness and accuracy by learning to defer,” arXiv:1711.06664, 2018.

[13] H. Mozannar and D. Sontag, “Consistent estimators for learning to defer to an expert,” arXiv:2006.01862, 2020.

[14] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, “Learning word vectors for sentiment analysis,” in *Proc. 49th Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2011, pp. 142–150.

[15] S. Teerapittayanon, B. McDanel, and H. T. Kung, “BranchyNet: Fast inference via early exiting from deep neural networks,” arXiv:1709.01686, 2017.

[16] S. Teerapittayanon, B. McDanel, and H. T. Kung, “Distributed deep neural networks over the cloud, the edge and end devices,” arXiv:1709.01921, 2017.

[17] J. Huang, S. Park, N. Paoletti, and O. Simeone, “Reliable inference in edge-cloud model cascades via conformal alignment,” arXiv:2510.17543, 2025.

[18] V. Rajapakse, I. Karunanayake, and N. Ahmed, “Intelligence at the extreme edge: A survey on reformable TinyML,” *ACM Computing Surveys*, 2023, doi: 10.1145/3583683.

[19] P. Raghuwanshi, V. Bhatia, S. Iyer, M. Latva-aho, and O. L. A. López, “TinyML for on-device and edge analytics in wireless networks: A survey of deployments, opportunities, and concept-drift mitigation,” arXiv:2606.30843, 2026.

[20] S. Jahanbazi, M. Ashraf, L. De Strycker, J. Famaey, and O. L. A. López, “Energy-aware multi-exit TinyML for smart zero-energy devices,” arXiv:2603.08047, 2026.

[21] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” arXiv:1906.02243, 2019.

[22] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” arXiv:1907.10597, 2019.

[23] T. Tambe *et al.*, “EdgeBERT: Sentence-level energy optimizations for latency-aware multi-task NLP inference,” arXiv:2011.14203, 2021.

[24] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, “Well-read students learn better: On the importance of pre-training compact models,” arXiv:1908.08962, 2019.

[25] K. Weinberger, A. Dasgupta, J. Attenberg, J. Langford, and A. Smola, “Feature hashing for large scale multitask learning,” arXiv:0902.2206, 2009.

[26] S. Wang and C. Manning, “Baselines and bigrams: Simple, good sentiment and topic classification,” in *Proc. 50th Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2012, pp. 90–94.

[27] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, “Bag of tricks for efficient text classification,” arXiv:1607.01759, 2016.

[28] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive label errors in test sets destabilize machine learning benchmarks,” arXiv:2103.14749, 2021.

[29] S. Ding *et al.*, “ERNIE-Doc: A retrospective long-document modeling transformer,” arXiv:2012.15688, 2021.

[30] S. Golchin and M. Surdeanu, “Data contamination quiz: A tool to detect and estimate contamination in large language models,” arXiv:2311.06233, 2025.

[31] E. Biswas, “IMDb review dataset,” Kaggle, CC0. [Online]. Available: https://www.kaggle.com/datasets/ebiswas/imdb-review-dataset

[32] Y. Hou *et al.*, “Bridging language and items for retrieval and recommendation,” arXiv:2403.03952, 2024.

[33] Y. Liu *et al.*, “RoBERTa: A robustly optimized BERT pretraining approach,” arXiv:1907.11692, 2019.

[34] C. Sun, X. Qiu, Y. Xu, and X. Huang, “How to fine-tune BERT for text classification?” arXiv:1905.05583, 2019.

[35] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, “ELECTRA: Pre-training text encoders as discriminators rather than generators,” arXiv:2003.10555, 2020.

[36] P. He, J. Gao, and W. Chen, “DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing,” arXiv:2111.09543, 2021.